

SRP Project: Optimising run production in a baseball batting order

Lily Neville

Intro/Structure of the game:

In Baseball, a batting lineup is the order in which players hit. A batting lineup has 9 players. Traditionally this order is roughly organised from the “best” hitter to the “worst” hitter. The order in which players go up to bat can be vital towards winning a game. There has been a lot of research on this topic for many years and as times go on, different strategies have been adopted based on new statistics and research.

A baseball game is divided into 9 innings. One inning consists of both teams going up to bat and playing defence. This is known as the top and bottom of the inning. For example if Team 1 goes up to bat in the top of the first inning and plays defence in the bottom of the inning, then Team 2 would play defence in the top of the inning and bat in the bottom of the inning. An inning ends once there are three outs. It carries on like this for the remainder of the game. The winner is the team that scores the most runs (points). You score a run by running all the bases and getting back to home plate.

The batting order is continuous throughout the game. For example, if team 1 sent 5 batters up to hit in the first inning, then the sixth batter would hit first in the second inning.

In this optimization problem, I would like to maximise the number of runs scored based on the ordering of the batting lineup. I will run the Monte Carlo simulation on every possible lineup to see which lineup produces the most runs.

Some Important Baseball Statistics:

- Batting Average: Percent of the time you get a hit when you go up to bat. Mathematically, this is measured as $\frac{h}{ab}$ where h is the number of hits and ab is the number of at bats a player has excluding walks.
- Single(1B): A hit that results in the batter reaching first base.
- Double(2B): A hit that results in the batter reaching second base.
- Triple(3B): A hit that results in the batter reaching third base.
- Home Run (HR): A hit that results in the batter scoring.
- Walk (BB): If a pitcher throws 4 balls to the batter, the player automatically goes to first base. A ball is a pitch out of the strike zone and therefore very hard to hit.
- On-Base-Percentage(OBP): This is the percentage of the time you get on base excluding reaching by a fielding error. This could include hits, walks, or hit by pitch. In other words, this is batting average plus percent of time a player walks or gets hit by a pitch. Traditionally, the batter first in the batting order has the highest OBP.

- Slugging Percentage: Total number of bases a player records per at bat. This is a good metric to measure a batter's power (how far they can hit the ball). Mathematically, this is measured $\frac{b}{ab}$, where b is the total number of bases and ab is the total number of at bats. When I say bases this refers to triples, doubles, singles, home runs. A single is one base, a double is 2 bases, etc.
- On-Base plus slugging (OPS): OBP + Slugging percentage. This statistic is also a good metric to measure power. Traditionally, players with higher OPS hit in the third and fourth spot in the batting lineup.
- Runs Batted In (RBI): This is the number of runs a player is responsible for. An RBI is awarded if when a player is up to bat they hit the ball or reach base in some way that brings a run home.

Sabermetrics:

The proper definition for sabermetrics is analytics and statistics that are developed through observation.

In recent years, sabermetrics has become a much more relevant way of doing things. After a lot of research, simulations, and reports, this is slowly becoming the norm throughout the baseball world. Sabermetrics are useful because they estimate a player's run production, speed, etc. In an ideal model, sabermetrics would be used to help determine a batting order, however, I will be using more basic statistics in my problem.

Constraints:

- No player may appear more than once in the lineup
- Only one player per lineup slot
- Players may only be selected from given data set
- Every Player must be in the lineup

Method/Data:

This is the data that I have used in my code.

Player	PA	1B	2B	3B	HR	BB	avg
Player a	100	15	10	3	4	10	0.355
Player b	107	24	5	0	1	23	0.357
Player c	94	13	3	1	0	12	0.207

Player d	101	14	16	5	8	15	0.500
Player e	99	20	3	1	1	9	0.278
Player f	100	16	1	0	0	4	0.177
Player g	102	14	17	3	6	10	0.435
Player h	98	12	6	4	3	12	0.291
Player i	96	13	6	1	3	6	0.256

- Based on data of 9 person lineup, have each person in the lineup be part of a list format
- I will run a “simulation” game on every possible lineup that uses the Monte Carlo Simulation and players' probability of getting a hit to estimate the number of runs this lineup would get. I will do this by doing the following:
 - Starting with player 1, they have a 25% chance of getting to first base ($\frac{1B+BB}{PA}$), a 10% chance of getting to second base ($\frac{2B}{PA}$), a 3% chance of getting to third base ($\frac{3B}{PA}$), a 4% chance of getting a home run ($\frac{HR}{PA}$), and a 58% chance of getting out ($100 - 10 - 25 - 4 - 3$).
 - I will run a random number generator that produces a number between 0-1, if that number is between 0-0.25, player 1 goes to first base, etc.
 - When the next batter comes up there will be a variable keeping track of which bases have players on them.
 - For simplicity I will assume the following: If player 1 is on 1st, player 2 must hit a double, triple or homerun in order for player 1 to score. If Player 1 is on second or third, Player 2 can have any hit and player 1 will score.
 - This cycle will continue until there are three outs
 - The next inning is simulated until all nine innings have been played.
 - The number of runs scored will be stored. This number is r .
- For each lineup I will run the previous procedure 1000 times and take the average number of runs that lineup produces.
- After a simulation, the lineup will switch up the order by doing the following:
 - The first and second slot in the lineup switch position, on the next simulation, first and third, etc. After the first slot has swapped with every position, the second slot switches with the third slot, etc.
 - If a lineup is found to have a higher run production than the current r , this number becomes the new r . Otherwise, r does not change. When r changes, I will store the lineup which produces the new r .

Objective Function:

- I am trying to maximise runs scored using the Monte Carlo Simulation.

Resources:

- https://digitalcommons.lib.uconn.edu/cgi/viewcontent.cgi?article=1967&context=srhonors_theses
- <https://ieeexplore.ieee.org/abstract/document/4736404>
- https://www.researchgate.net/profile/Emanuel-Schorsch/publication/328529782_Baseball_Lineup_Optimization/links/5bd2a90b4585150b2b877139/Baseball-Lineup-Optimization.pdf
- https://www.iacss.org/fileadmin/user_upload/IJCSS_FullPaper/Vol12_2013_Ed1/IJCSS-Volume12_2013_Edition1_Robinson.pdf